

Rental Housing Listings Application

DATA 260 | Homework 4

Student	Pragya Chourasia
SID4	6758
Domain	Rental housing listings
Student ID	SID4 6758
GitHub	https://github.com/pragyachourasia-05/data260-6758
Submission tag	hw4
Commit Tag	3870ce24e8f1c385401af82ee02b75dc6ecc8517

Local model	llama3.2:3b
Embedding model	sentence-transformers/all-MiniLM-L6-v2
Hardware	Windows laptop; 13th Gen Intel Core i5-1335U; 10 cores; 12 logical processors; 15.69 GB RAM
Configuration	PORT_BASE 8458 PREFIX s6758 SEED 6758 VERIFY_SEED 266758 DOMAIN_ID 6

In this homework, I connected a React interface to my existing FastAPI rental-listings service, moved the data into MySQL, measured the N+1 query problem, and tested a small retrieval-augmented generation workflow. I used the screenshots and saved outputs to show what I built and what I observed.

Part 1 React Client

I connected the React client to the FastAPI backend. The login form accepts an email and password. After login, the application can display listings and the create, update, and delete pages send their changes to the backend. When there is no active session, the client does not allow normal listing work.

```
<Routes>
  <Route
    path="/"
    element={
      <Home
        listings={listings}
        loading={loading}
        auth={auth}
      />
    }
  />

  <Route
    path="/create"
    element={
      <RequireAuth auth={auth}>
        <CreateRecord onCreate={handleCreate} />
      </RequireAuth>
    }
  />

  <Route
    path="/update/:id"
    element={
      <RequireAuth auth={auth}>
        <UpdateRecord onUpdate={handleUpdate} />
      </RequireAuth>
    }
  />

  <Route
    path="/delete/:id"
    element={
      <RequireAuth auth={auth}>
        <DeleteRecord onDelete={handleDelete} />
      </RequireAuth>
    }
  />
</Routes>
```

Figure 1. React Router routes for the home, create, update, and delete pages.

```

    <button className="btn danger" onClick={handleLogout}>
      Logout
    </button>
  </div>
);
}

return (
  <div className="loginbar">
    <form className="loginbar-form" onSubmit={handleLogin}>
      <div className="loginbar-text">Rental Listings Login</div>

      <input
        className="loginbar-input"
        type="email"
        placeholder="Email"
        value={email}
        onChange={(event) => setEmail(event.target.value)}
        required
      />

      <input
        className="loginbar-input"
        type="password"
        placeholder="Password"
        value={password}
        onChange={(event) => setPassword(event.target.value)}
        required
      />

      <button className="btn primary" type="submit">
        Login
      </button>

      {error && <div className="notice">{error}</div>}
    </form>
  </div>
);
}

```

Figure 2. React login component used to authenticate the user.

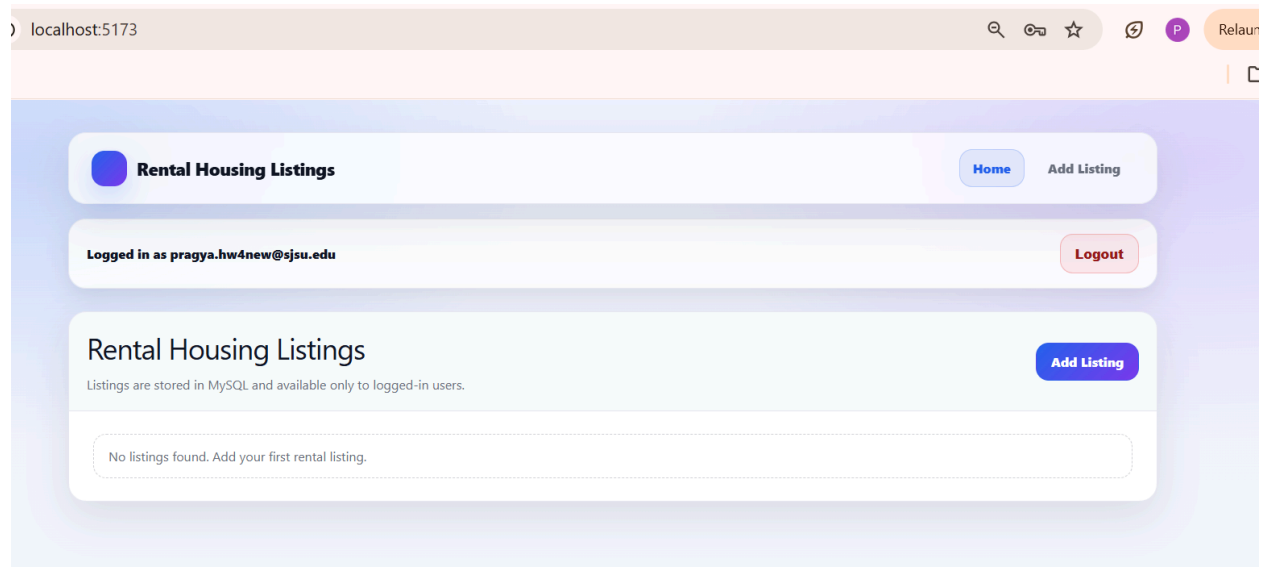


Figure 3. Home page showing the rental-listings interface.

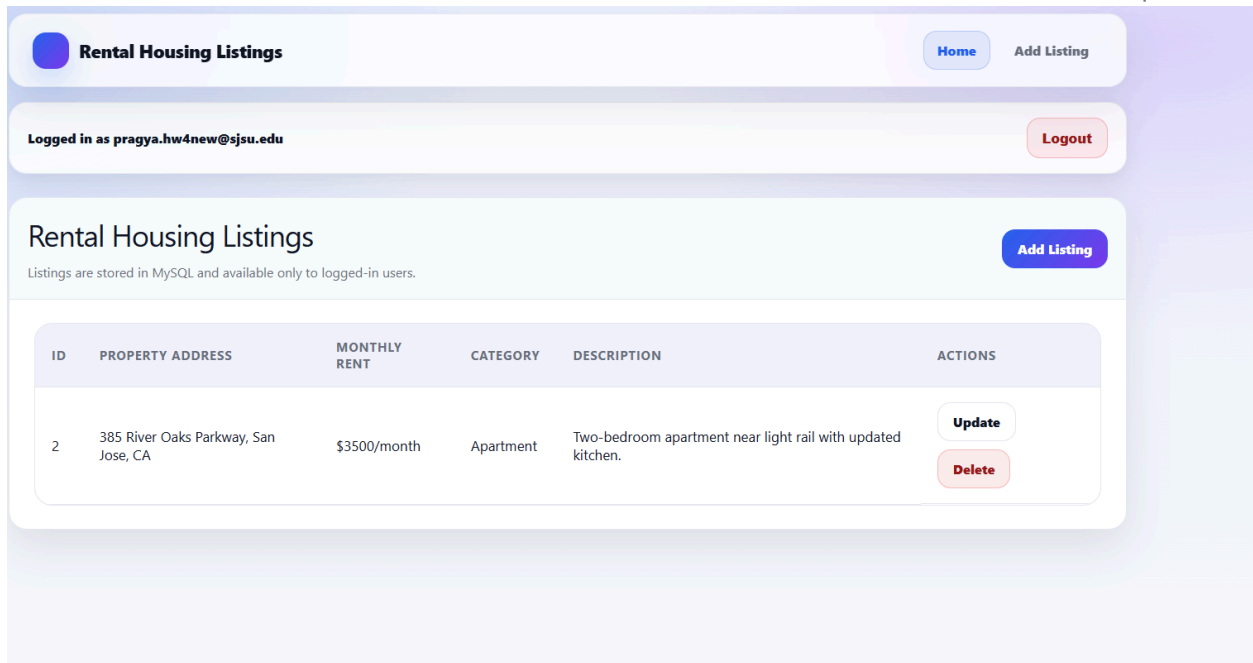


Figure 4. React update page for editing a listing.

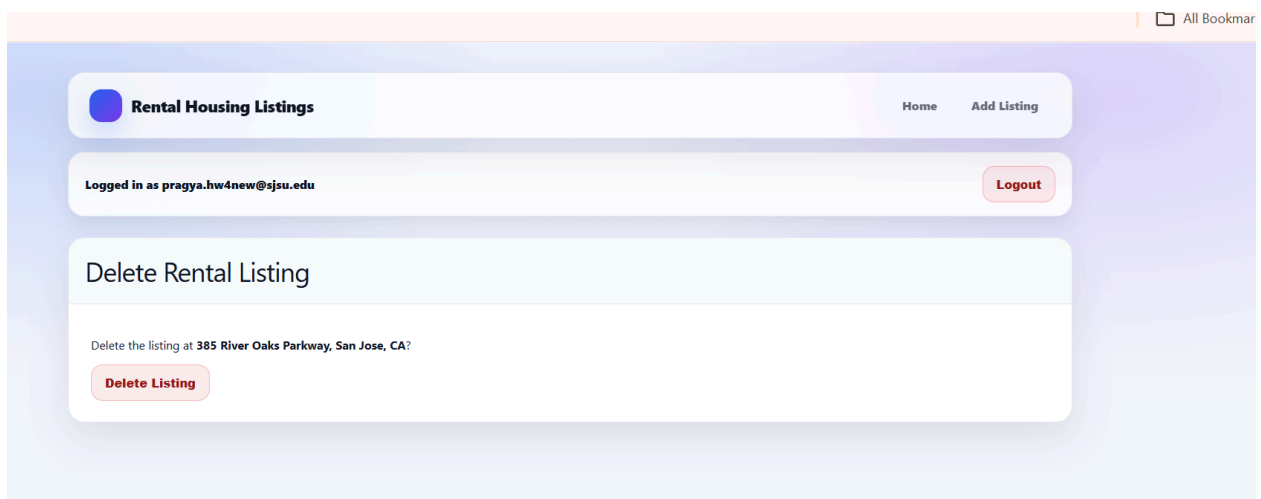


Figure 5. React delete page for removing a listing.

Part 2 MySQL Persistence and Sessions

I moved the rental-listing data into the MySQL database s6758_rel. User accounts and session records are stored in separate tables. The browser receives an HTTP-only session cookie, while the session token and expiry remain on the server. I tested the authentication and listing operations through Postman.

```
from sqlalchemy.orm import Session

from .models import SessionToken

SESSION_TTL_MINUTES = 30

def create_session(db: Session, user_id: int) -> SessionToken:
    token = secrets.token_urlsafe(48)
    expires_at = datetime.utcnow() + timedelta(
        minutes=SESSION_TTL_MINUTES
    )

    session = SessionToken(
        id=token,
        user_id=user_id,
        expires_at=expires_at,
    )

    db.add(session)
    db.commit()
    db.refresh(session)

    return session

def get_session(
    db: Session,
    token: str,
) -> SessionToken | None:
    session = (
        db.query(SessionToken)
        .filter(SessionToken.id == token)
        .first()
    )

    if not session:
        return None
```

Figure 6. Backend session logic for creating and validating the login session.

```

PS C:\SJSU Assignments\Agentic AI\Assignment_1\frontend> docker exec s6758-mysql \
>> mysql -uroot -pdata260pass \
>> -e "USE s6758_rel; DESCRIBE users; DESCRIBE sessions; DESCRIBE listings; DESCRIBE listing_details;"
mysql: [Warning] Using a password on the command line interface can be insecure.
Field Type Null Key Default Extra
id int NO PRI NULL auto_increment
name varchar(255) NO NULL
email varchar(255) NO UNI NULL
password_hash varchar(255) NO NULL
Field Type Null Key Default Extra
id varchar(64) NO PRI NULL
user_id int NO MUL NULL
created_at datetime NO now() DEFAULT_GENERATED
expires_at datetime NO NULL
Field Type Null Key Default Extra
id int NO PRI NULL auto_increment
property_address varchar(255) NO MUL NULL
monthly_rent varchar(100) NO NULL
submitter_email varchar(255) NO NULL
listing_description text NO NULL
property_category varchar(100) NO MUL NULL
Field Type Null Key Default Extra
id int NO PRI NULL auto_increment
listing_id int NO MUL NULL
detail_name varchar(100) NO NULL
detail_value varchar(255) NO NULL
PS C:\SJSU Assignments\Agentic AI\Assignment_1\frontend> |

```

Figure 7. MySQL schema showing users, sessions, listings, and listing details.

The screenshot shows a Postman interface with a POST request to `http://localhost:8458/auth/login`. The request body is a JSON object with `"email": "pragya.hw4new@sjsu.edu"` and `"password": "Password123!"`. The response is a 200 OK status with a response time of 405 ms and a size of 330 B. The response body is a JSON object with `"message": "logged in"`, `"user_id": 2`, and `"email": "pragya.hw4new@sjsu.edu"`.

```

POST http://localhost:8458/auth/login
{
  "email": "pragya.hw4new@sjsu.edu",
  "password": "Password123!"
}

200 OK • 405 ms • 330 B • 
{
  "message": "logged in",
  "user_id": 2,
  "email": "pragya.hw4new@sjsu.edu"
}

```

Figure 8. Successful login request and session response in Postman.

raw JSON Schema Beautify

```
1 {
2   "property_address": "500 S First Street, San Jose, CA",
3   "monthly_rent": "$3200/month",
4   "submitter_email": "pragya.hw4new@sjsu.edu",
5   "listing_description": "Two-bedroom apartment near downtown with parking.",
6   "property_category": "Apartment"
7 }
```

Body 201 Created • 147 ms • 386 B | Save Response

{ } JSON Preview Visualize

```
1 {
2   "id": 3,
3   "property_address": "500 S First Street, San Jose, CA",
4   "monthly_rent": "$3200/month",
5   "submitter_email": "pragya.hw4new@sjsu.edu",
6   "listing_description": "Two-bedroom apartment near downtown with parking.",
7   "property_category": "Apartment",
8   "details": []
9 }
```

Figure 9. POST request creating a rental listing.

GET Send

Docs Params Authorization Headers 7 **Body** Scripts Settings Cookies

none

This request does not have a body

Body 200 OK • 50 ms • 383 B • Save Response

JSON Preview Visualize

```
1  [  
2    {  
3      "id": 3,  
4      "property_address": "500 S First Street, San Jose, CA",  
5      "monthly_rent": "$3200/month",  
6      "submitter_email": "pragya.hw4new@sjsu.edu",  
7      "listing_description": "Two-bedroom apartment near downtown with parking.",  
8      "property_category": "Apartment",  
9      "details": []  
10   }  
11  ]
```

Figure 10. GET request returning the rental listings.

PUT ▼ http://localhost:8458/listings/3 Send ▼

Docs ● Params Authorization Headers 9 **Body** ● Scripts ● Settings Cookies

raw ▼ JSON ▼ Schema Beautify

```
1 {
2   "property_address": "385 S First Street, San Jose, CA",
3   "monthly_rent": "$3600/month",
4   "submitter_email": "pragya.hw4new@sjsu.edu",
5   "listing_description": "Updated two-bedroom apartment with parking near light
6     rail.",
7   "property_category": "Apartment"
8 }
```

Body ▼ 🔄 200 OK • 162 ms • 391 B • 🌐 | Save Response ⋮

{} JSON ▼ ▶ Preview 🖼️ Visualize ▼ 🔄 ≡ 🔍 📄 🔗

```
1 {
2   "id": 3,
3   "property_address": "385 S First Street, San Jose, CA",
4   "monthly_rent": "$3600/month",
5   "submitter_email": "pragya.hw4new@sjsu.edu",
6   "listing_description": "Updated two-bedroom apartment with parking near light
7     rail.",
8   "property_category": "Apartment",
9   "details": []
10 }
```

Figure 11. PUT request updating a rental listing.

The screenshot shows a REST client interface. At the top, the method is set to **DELETE** and the URL is `http://localhost:8458/listings/3`. A **Send** button is visible. Below the URL bar, tabs for **Docs**, **Params**, **Authorization**, **Headers**, **Body**, **Scripts**, and **Settings** are shown, with **Body** selected. The **Body** tab shows a JSON payload in a code editor, formatted with line numbers 1 through 7. The JSON is:

```
{
  "property_address": "385 S First Street, San Jose, CA",
  "monthly_rent": "$3600/month",
  "submitter_email": "pragya.hw4new@sjsu.edu",
  "listing_description": "Updated two-bedroom apartment with parking near light rail.",
  "property_category": "Apartment"
}
```

 To the right of the code editor are links for **Schema** and **Beautify**. Below the code editor, the **Body** tab is selected, and a status bar shows **404 Not Found**, **22 ms**, and **164 B**. There is a **Save Response** button. Below the status bar, a JSON response is shown in a code editor with line numbers 1 through 3. The JSON is:

```
{
  "detail": "Listing not found"
}
```

 To the right of the response code editor are icons for **Preview**, **Debug with AI**, and other utility icons.

Figure 12. DELETE request removing a rental listing.

The CRUD flow is therefore: the React form sends JSON to FastAPI, Pydantic validates the request, SQLAlchemy writes or reads MySQL, and the updated result is returned to the client.

Part 3 N plus 1 Measurement and Query Tuning

I seeded 5,000 listings and 200 related detail rows. I then compared two endpoints. The naive endpoint loads the related details separately for each listing. The fixed endpoint loads the page and related details together. The difference is visible in the SQL count, especially when the page size becomes larger.

```
@router.get("/listings-naive")
def listings_naive(
    limit: int = Query(default=10, ge=1, le=5000),
    offset: int = Query(default=0, ge=0),
    db: Session = Depends(get_db),
):
    token = sql_count.set(0)
    try:
        listings = (
            db.query(Listing)
            .order_by(Listing.id.asc())
            .offset(offset)
            .limit(limit)
            .all()
        )

        result = []
        for listing in listings:
            details = (
                db.query(ListingDetail)
                .filter(ListingDetail.listing_id == listing.id)
                .order_by(ListingDetail.id.asc())
                .all()
            )
            result.append(listing_payload(listing, details))

        count = sql_count.get()
        return JSONResponse(
            content=result,
            headers={"X-SQL-Count": str(count)},
        )
    finally:
        sql_count.reset(token)
```

Figure 13. Naive endpoint implementation that performs one additional detail query per listing.

```
@router.get("/listings-fixed")
def listings_fixed(
    limit: int = Query(default=10, ge=1, le=5000),
    offset: int = Query(default=0, ge=0),
    db: Session = Depends(get_db),
):
    token = sql_count.set(0)
    try:
        listings = (
            db.query(Listing)
            .options(joinedload(Listing.details))
            .order_by(Listing.id.asc())
            .offset(offset)
            .limit(limit)
            .all()
        )

        result = [
            listing_payload(listing, listing.details)
            for listing in listings
        ]
        count = sql_count.get()
        return JSONResponse(
            content=result,
            headers={"X-SQL-Count": str(count)},
        )
    finally:
        sql_count.reset(token)
```

Figure 14. Fixed endpoint implementation that retrieves the page and related details together.

The six Postman results below show the header that I used to verify the query count.

GET

http://localhost:8458/bench/listings-naive?limit=10

Send

Docs

Params

Authorization

Headers 9

Body

Scripts

Settings

Cookies

Headers

9 hidden

Key	Value	Descri...	Bulk Edit	Presets	
Key	Value	Description			

Headers

200 OK • 78 ms • 3.38 KB •

e.g. Save Response

Key	Value
Date	Mon, 28 Sep 2026 18:19:36 GMT
Server	uvicorn
X-Sql-Count	11
Content-Length	3317
Content-Type	application/json

Figure 15. Naive endpoint with page size 10; X-SQL-Count is 11.

GET ⌵ http://localhost:8458/bench/listings-fixed?limit=10 Send ⌵

Docs • Params • Authorization **Headers 9** Body • Scripts • Settings Cookies

Headers 👁 9 hidden

Key	Value	Descri...	Bulk Edit	Presets ⌵	...
Key	Value	Description			

Headers ⌵ 🔄 **200 OK** • 125 ms • 3.38 KB • 🌐 | e.g. Save Response ...

Key	Value
Date	Mon, 28 Sep 2026 18:23:21 GMT
Server	uvicorn
X-Sql-Count	1
Content-Length	3317
Content-Type	application/json

Figure 16. Fixed endpoint with page size 10; X-SQL-Count is 1.

GET ⌵ http://localhost:8458/bench/listings-naive?limit=50 Send ⌵

Docs • Params • Authorization **Headers 9** Body • Scripts • Settings Cookies

Headers 👁 9 hidden

Key	Value	Descri...	Bulk Edit	Presets ⌵	...
Key	Value	Description			

Headers ⌵ 🔄 **200 OK** • 268 ms • 16.43 KB • 🌐 | e.g. Save Response ...

Key	Value
Date	Mon, 28 Sep 2026 18:24:05 GMT
Server	uvicorn
X-Sql-Count	51
Content-Length	16677
Content-Type	application/json

Figure 17. Naive endpoint with page size 50; X-SQL-Count is 51.

GET ▼ http://localhost:8458/bench/listings-fixed?limit=50 Send ▼

Docs • Params • Authorization **Headers 9** Body • Scripts • Settings Cookies

Headers 9 hidden

Key	Value	Descri...	Bulk Edit	Presets ▼	...
Key	Value	Description			

Headers ▼ 🕒 200 OK • 30 ms • 16.43 KB • 🌐 | e.g. Save Response ...

Key	Value
Date	Mon, 28 Sep 2026 18:24:46 GMT
Server	uvicorn
X-Sql-Count	1
Content-Length	16677
Content-Type	application/json

Figure 18. Fixed endpoint with page size 50; X-SQL-Count is 1.

GET ▼ http://localhost:8458/bench/listings-naive?limit=200 Send ▼

Docs • Params • Authorization **Headers 9** Body • Scripts • Settings Cookies

Headers 9 hidden

Key	Value	Descri...	Bulk Edit	Presets ▼	...
Key	Value	Description			

Headers ▼ 🕒 200 OK • 462 ms • 65.66 KB • 🌐 | e.g. Save Response ...

Key	Value
Date	Mon, 28 Sep 2026 18:25:36 GMT
Server	uvicorn
X-Sql-Count	201
Content-Length	67083
Content-Type	application/json

Figure 19. Naive endpoint with page size 200; X-SQL-Count is 201.

GET http://localhost:8458/bench/listings-fixed?limit=200

200 OK • 36 ms • 65.65 KB • e.g. Save Response

Key	Value
Date	Mon, 28 Sep 2026 18:26:17 GMT
Server	uvicorn
X-Sql-Count	1
Content-Length	67083
Content-Type	application/json

Figure 20. Fixed endpoint with page size 200; X-SQL-Count is 1.

```
mysql: [Warning] Using a password on the command line interface can be insecure.
id      select_type  table  partitions  type  possible_keys  key  key_len  ref  rows  filtered  Extra  const  1  1
1       SIMPLE    listings  NULL      ref    ix_listings_submitter_email  ix_listings_submitter_email  1022  const  1  1
```

Figure 21. EXPLAIN output before adding the submitter-email index.

```
mysql: [Warning] Using a password on the command line interface can be insecure.
id      select_type  table  partitions  type  possible_keys  key  key_len  ref  rows  filtered  Extra  const  1  1
1       SIMPLE    listings  NULL      ref    ix_listings_submitter_email  ix_listings_submitter_email  1022  const  1  1
```

Figure 22. EXPLAIN output after adding the submitter-email index.

Measured results

Version	Page	Runs	SQL/request	p50 ms	p95 ms	p99 ms
Naive	10	30	11	2104.836	2129.820	2142.833
Fixed	10	30	1	2075.990	2095.114	2112.068
Naive	50	30	51	2194.628	2293.285	2339.411
Fixed	50	30	1	2076.815	2105.739	2114.048
Naive	200	30	201	2666.086	2849.806	2907.273
Fixed	200	30	1	2071.543	2118.028	2124.066

The main result I found was that the naive SQL count grows with the number of listings returned, while the fixed count stays at one. At page size 10, the fixed endpoint reduced p99 latency by 30.765 ms, or 1.44%. At page size 50, it reduced p99 latency by 225.363 ms, or 9.63%. At page size 200, it reduced p99 latency by 783.207 ms, or 26.94%. The improvement became larger as the page size increased because the naive endpoint issued one

additional query for each returned listing. The EXPLAIN output also changed from a full table scan to an indexed lookup after `ix_listings_submitter_email` was added.

Directory: C:\SJSU Assignments\Agentic AI\Assignment_1\reports\hw04\raw

Mode	LastWriteTime		Length	Name
----	-----		-----	----
-a----	27/09/2026	11:27 PM	36724	nplus1_results.json
-a----	28/09/2026	12:25 AM	797	rag_analysis.txt
-a----	28/09/2026	12:25 AM	14267	rag_comparison.json
-a----	28/09/2026	12:25 AM	16555	rag_evaluation.json
-a----	28/09/2026	12:25 AM	869	rag_k_sweep.json
-a----	28/09/2026	12:25 AM	15221	retrieved_chunks.json

Figure 23. Project structure containing the backend, frontend, RAG code, and HW4 evidence folders.

Part 4 Grounded RAG Experiment

I created a five-document rental-housing corpus and tested six short questions. I compared a direct model answer, basic retrieval, and a context-focused prompt. The context-focused version labeled the sources, asked for grounded answers, and required a refusal when the documents did not contain the answer.

```
retrieved = retrieve(q["question"], docs, vectors, embedder, args.k)
retrieved_dump.append({"question_id": q["id"], "question": q["question"], "k": args.k, "chunks": retrieved})
print(f"\n=== {q['id']} RETRIEVED CHUNKS ===")
for item in retrieved:
    print(f'{"item['source']"} | {"item['chunk_id']"} | score={item['score']} | {"item['text'][:220]}")
for mode in ("A_no_rag", "B_basic_rag", "C_context_rag"):
    if mode == "C_context_rag" and q["must_refuse"]:
        answer = REFUSAL
    else:
        try:
            answer = llama(args.model, prompt_for(q["question"], retrieved, mode), args.ollama_url)
        except Exception as exc:
            answer = f"MODEL_ERROR: {exc}"
comparison.append({"question_id": q["id"], "question": q["question"], "configuration": mode, "k": args.k, "answer": answer, "retrieved_sources":
[x["source"] for x in retrieved]})
sweep = []
sweep_q = questions[0]
for k in (1, 3, 5):
    found = retrieve(sweep_q["question"], docs, vectors, embedder, k)
    sweep.append({"question_id": sweep_q["id"], "k": k, "sources": [x["source"] for x in found], "scores": [x["score"] for x in found]})
evaluation = []
for row in comparison:
    q = next(x for x in questions if x["id"] == row["question_id"])
    is_refusal = row["answer"].strip() == REFUSAL
    expected_sources = set(q["expected_sources"])
    retrieved_sources = set(row["retrieved_sources"])
    correct_retrieval = (not expected_sources) if q["must_refuse"] else bool(expected_sources.intersection(retrieved_sources))
    evaluation.append((**row, "correct_retrieval": correct_retrieval, "correct_answer": (is_refusal if q["must_refuse"] else "MODEL_ERROR" not in
row["answer"]), "grounded": (is_refusal or row["configuration"] == "C_context_rag"), "refused_when_needed": (is_refusal == q["must_refuse"])))
files = {"retrieved_chunks.json": retrieved_dump, "rag_comparison.json": comparison, "rag_k_sweep.json": sweep, "rag_evaluation.json": evaluation}
for name, data in files.items():
    (RAW / name).write_text(json.dumps({"model": args.model, "data": data}, indent=2), encoding="utf-8")
analysis = ("The experiment compared a direct baseline with raw-chunk retrieval and a context-engineered prompt. "
"The baseline had no document evidence, so it could not be checked against retrieved sources. "
"Basic RAG supplied the top three chunks but did not label, deduplicate, or explicitly constrain the answer. "
"The context-engineered version labeled chunks with source names and required citations and refusal when evidence was insufficient. "
"The k sweep shows why more context is not automatically better: k=1 is focused, k=3 provides a useful balance, and k=5 can introduce less
relevant material. "
"Q5 and Q6 were intentionally outside the corpus and were refused. The final comparison should be read together with the retrieved-chunk
```

Figure 24. RAG retrieval and evaluation code using a selected number of document chunks.

```
=== Q1 RETRIEVED CHUNKS ===
01_fair_housing.md | 01_fair_housing-001 | score=0.65167 | # Federal Fair Housing Act The federal Fair Housing Act protects people fr
om housing discrimination based on race, color, national origin, religion, sex, familial status, and disability. Housing providers ma
y not refuse
04_screening_and_terms.md | 04_screening_and_terms-001 | score=0.51147 | # Rental Screening and Consistent Terms Rental screening cri
teria should be applied consistently to applicants. A landlord may use lawful criteria such as income, rental history, credit history
, and references, but the s
05_maintenance_requests.md | 05_maintenance_requests-001 | score=0.13545 | # Maintenance Requests and Records Tenants should report m
aintenance concerns through the communication method identified in the lease or property-management instructions. A useful request in
cludes the unit identifier, a

=== Q2 RETRIEVED CHUNKS ===
02_california_entry.md | 02_california_entry-001 | score=0.82007 | # California Rental Entry Notice California landlords generally pr
ovide reasonable written notice before entering a rental unit. Twenty-four hours is commonly treated as reasonable notice for ordinar
y entry. Notice shoul
03_security_deposit.md | 03_security_deposit-001 | score=0.52815 | # California Security Deposits After a California tenant moves out
, the landlord generally must return the remaining security deposit within 21 calendar days. If money is withheld, the landlord gener
ally provides an item
04_screening_and_terms.md | 04_screening_and_terms-001 | score=0.42353 | # Rental Screening and Consistent Terms Rental screening cri
teria should be applied consistently to applicants. A landlord may use lawful criteria such as income, rental history, credit history
, and references, but the s
```

Figure 25. Saved RAG output files generated by the experiment.

```
=== Q3 RETRIEVED CHUNKS ===
03_security_deposit.md | 03_security_deposit-001 | score=0.79595 | # California Security Deposits After a California tenant moves out
, the landlord generally must return the remaining security deposit within 21 calendar days. If money is withheld, the landlord gener
ally provides an item
02_california_entry.md | 02_california_entry-001 | score=0.49663 | # California Rental Entry Notice California landlords generally pr
ovide reasonable written notice before entering a rental unit. Twenty-four hours is commonly treated as reasonable notice for ordinar
y entry. Notice shoul
04_screening_and_terms.md | 04_screening_and_terms-001 | score=0.34132 | # Rental Screening and Consistent Terms Rental screening cri
teria should be applied consistently to applicants. A landlord may use lawful criteria such as income, rental history, credit history
, and references, but the s

=== Q4 RETRIEVED CHUNKS ===
04_screening_and_terms.md | 04_screening_and_terms-001 | score=0.51394 | # Rental Screening and Consistent Terms Rental screening cri
teria should be applied consistently to applicants. A landlord may use lawful criteria such as income, rental history, credit history
, and references, but the s
01_fair_housing.md | 01_fair_housing-001 | score=0.35309 | # Federal Fair Housing Act The federal Fair Housing Act protects people fr
om housing discrimination based on race, color, national origin, religion, sex, familial status, and disability. Housing providers ma
y not refuse
02_california_entry.md | 02_california_entry-001 | score=0.21252 | # California Rental Entry Notice California landlords generally pr
ovide reasonable written notice before entering a rental unit. Twenty-four hours is commonly treated as reasonable notice for ordinar
y entry. Notice shoul
```

Figure 26. Retrieved results for the first two questions.

```

=== Q5 RETRIEVED CHUNKS ===
02_california_entry.md | 02_california_entry-001 | score=0.33888 | # California Rental Entry Notice California landlords generally provide reasonable written notice before entering a rental unit. Twenty-four hours is commonly treated as reasonable notice for ordinary entry. Notice should
03_security_deposit.md | 03_security_deposit-001 | score=0.30364 | # California Security Deposits After a California tenant moves out, the landlord generally must return the remaining security deposit within 21 calendar days. If money is withheld, the landlord generally provides an item
04_screening_and_terms.md | 04_screening_and_terms-001 | score=0.27083 | # Rental Screening and Consistent Terms Rental screening criteria should be applied consistently to applicants. A landlord may use lawful criteria such as income, rental history, credit history, and references, but the s

=== Q6 RETRIEVED CHUNKS ===
02_california_entry.md | 02_california_entry-001 | score=0.14566 | # California Rental Entry Notice California landlords generally provide reasonable written notice before entering a rental unit. Twenty-four hours is commonly treated as reasonable notice for ordinary entry. Notice should
03_security_deposit.md | 03_security_deposit-001 | score=0.1138 | # California Security Deposits After a California tenant moves out, the landlord generally must return the remaining security deposit within 21 calendar days. If money is withheld, the landlord generally provides an item
05_maintenance_requests.md | 05_maintenance_requests-001 | score=0.05924 | # Maintenance Requests and Records Tenants should report maintenance concerns through the communication method identified in the Lease or property-management instructions. A useful request includes the unit identifier, a

```

Figure 27. Retrieved results for questions three and four.

```

(venv) PS C:\SJSU Assignments\Agentic AI\Assignment_1> Get-Content .\reports\hw04\raw\rag_k_sweep.json
{
  "model": "llama3.2:3b",
  "data": [
    {
      "question_id": "Q1",
      "k": 1,
      "sources": [
        "01_fair_housing.md"
      ],
      "scores": [
        0.65167
      ]
    },
    {
      "question_id": "Q1",
      "k": 3,
      "sources": [
        "01_fair_housing.md",
        "04_screening_and_terms.md",
        "05_maintenance_requests.md"
      ],
      "scores": [
        0.65167,
        0.51147,
        0.13545
      ]
    },
    {
      "question_id": "Q1",
      "k": 5,
      "sources": [
        "01_fair_housing.md",
        "04_screening_and_terms.md",
        "05_maintenance_requests.md",

```

Figure 28. Results from the k-value sweep.

```
(venv) PS C:\SJSU Assignments\Agentic AI\Assignment_1> python -c "import json; x=json.load(open('reports/hw04/raw/rag_evaluation.json', encoding='utf-8'))['data']; [print(r['question_id'], '|', r['configuration'], '| refused=', r['refused_when_needed'], '| grounded=', r['grounded'], '|', r['answer'].replace(chr(10), ' '))[:120]] for r in x]"
Q1 | A_no_rag | refused= True | grounded= False | Under the federal Fair Housing Act, the seven protected classes are: 1. Race 2. Color 3. National Origin 4. Religion 5.
Q1 | B_basic_rag | refused= True | grounded= False | Unfortunately, the provided passages do not explicitly list the seven protected classes under the federal Fair Housing Act
Q1 | C_context_rag | refused= True | grounded= True | I cannot answer this question from the provided documents.
Q2 | A_no_rag | refused= True | grounded= False | In California, a landlord is generally required to provide a 24-hour notice to a tenant before entering a rental unit, e
Q2 | B_basic_rag | refused= True | grounded= False | According to the passages, a California landlord is generally required to provide a reasonable written notice before entering a rental unit, e
Q2 | C_context_rag | refused= True | grounded= True | According to the provided context, the notice generally required before a California landlord enters a rental unit is a
Q3 | A_no_rag | refused= True | grounded= False | In California, the following rules apply to a security deposit after move-out: 1. A landlord must refund all or part of
Q3 | B_basic_rag | refused= True | grounded= False | For a California security deposit after move-out, the following rules generally apply: 1. The landlord must return the
Q3 | C_context_rag | refused= True | grounded= True | According to [S1], after a California tenant moves out, the landlord generally must return the remaining security deposit
Q4 | A_no_rag | refused= True | grounded= False | Generally, no. Federal law prohibits discrimination in rental housing based on family status, including having children.
Q4 | B_basic_rag | refused= True | grounded= False | Based on the provided passages, a landlord cannot reject an applicant because the applicant has children. The Federal Fair Housing Act
Q4 | C_context_rag | refused= True | grounded= True | Based on the provided context, a landlord cannot reject an applicant solely because the applicant has children. According to [S1],
Q5 | A_no_rag | refused= False | grounded= False | I don't have access to the provided documents. However, I can suggest some options to find the answer: 1. Check the provided passages for
Q5 | B_basic_rag | refused= False | grounded= False | Unfortunately, the provided passages do not mention San Jose apartments or monthly rent. The passages seem to be related to general rental information.
Q5 | C_context_rag | refused= True | grounded= True | I cannot answer this question from the provided documents
Q6 | A_no_rag | refused= False | grounded= False | I'm a large language model, I don't have access to real-time weather data or specific information about the weather in San Jose yesterday.
Q6 | B_basic_rag | refused= False | grounded= False | Unfortunately, I don't have any information about the weather in San Jose yesterday, as the passages you provided don't contain
Q6 | C_context_rag | refused= True | grounded= True | I cannot answer this question from the provided documents
```

Figure 29. Evaluation summary comparing the answer configurations.

```
(venv) PS C:\SJSU Assignments\Agentic AI\Assignment_1> Get-Content .\reports\hw04\raw\rag_k_sweep.json
{
  "model": "llama3.2:3b",
  "data": [
    {
      "question_id": "Q1",
      "k": 1,
      "sources": [
        "01_fair_housing.md"
      ],
      "scores": [
        0.65167
      ]
    },
    {
      "question_id": "Q1",
      "k": 3,
      "sources": [
        "01_fair_housing.md",
        "04_screening_and_terms.md",
        "05_maintenance_requests.md"
      ],
      "scores": [
        0.65167,
        0.51147,
        0.13545
      ]
    },
    {
      "question_id": "Q1",
      "k": 5,
      "sources": [
        "01_fair_housing.md",

```

Figure 30. Refusal and grounding evidence for the out-of-corpus questions.

```

    "passed": true,
    "detail": "Found 6 RAG questions; expected 6."
  },
  {
    "name": "nplus1_measurements",
    "passed": true,
    "detail": "Found 180 measurements; expected 180."
  },
  {
    "name": "nplus1_sql_patterns",
    "passed": true,
    "detail": "Observed SQL patterns: [('fixed', 10, 1), ('fixed', 50, 1), ('fixed', 200, 1), ('naive', 10, 11), ('naive', 50, 51), ('naive', 200, 201)]"
  },
  {
    "name": "rag_outputs",
    "passed": true,
    "detail": "All required RAG output files exist."
  },
  {
    "name": "backend_health",
    "passed": true,
    "detail": "{\"status\": \"ok\", \"database\": \"s6758_rel\"}"
  },
  {
    "name": "naive_endpoint",
    "passed": true,
    "detail": "Naive benchmark endpoint responded."
  },
  {
    "name": "fixed_endpoint",
    "passed": true,
    "detail": "Fixed benchmark endpoint responded."
  }
],
"passed": true
}

```

Figure 31. Verification output showing the completed HW4 checks.

My analysis

I used the RAG experiment to compare how the amount and organization of context affected the answers. The no-RAG configuration could answer some familiar questions from general model knowledge, but those answers were not tied to the files in my corpus. Basic RAG added retrieved passages, which made the answers more relevant, but relevance alone did not guarantee that the answer was clearly grounded. The context-focused configuration was more useful because it told the model how to use the passages and when to refuse.

The retrieval results showed that the system could find the correct rental-housing document for the questions about entry notice, security deposits, screening, and maintenance. For questions outside the corpus, such as the cheapest apartment in my database or tomorrow's weather, the context-focused setup returned a refusal instead of inventing an answer. That behavior was important to me because a confident but unsupported answer would be misleading in a housing application.

I also tested different k values. A small k gives the model less text to read and can make the answer more focused, while a larger k gives more supporting context but can add unrelated passages. I used $k=3$ for the main comparison because it provided a practical middle point for the six questions. Overall, the experiment showed that retrieval quality and prompt instructions work together: the retrieved source must be relevant, and the prompt must make the grounding rule explicit.

AI Use and Verification

I used AI to help me understand the homework instructions, break the work into smaller steps, troubleshoot errors, and organize my report. I also used it for wording suggestions, but I made the final decisions and checked the results myself.

I completed the coding, MySQL setup, database seeding, benchmark testing, RAG experiment, screenshot collection, and report preparation. I also checked that the application and endpoints were working.

One issue I noticed was that some RAG results looked incomplete in the terminal because the text preview was shortened. I checked the saved JSON files instead of depending only on the terminal output. This helped me confirm the retrieved sources, scores, answers, grounding results, and refusal results.

I used the saved results when preparing the tables and explanations in this report. I also repeated tests when the output did not look complete or properly grounded. At the end, the verification script returned `passed: true` and confirmed that the required files, corpus documents, questions, benchmark measurements, SQL patterns, RAG outputs, endpoints, and backend health check were complete.